

Software Requirements Specification (SRS)

Project Title: End-to-End Sales Analytics Dashboard: Revenue Trends, Product Performance, and Customer Segmentation

1. Introduction

1.1 Purpose & Scope

This document specifies the software and analytical requirements for a comprehensive retail/B2B sales data project. The system will process historical sales transactions to evaluate revenue trends, identify top-performing products, assess regional sales health, and segment the customer base. The project covers the complete data lifecycle: extraction, relational database management, exploratory data analysis (EDA), predictive analytics/segmentation, and executive dashboarding.

1.2 Key Objectives

- Calculate core financial metrics including Gross Margin, Year-over-Year (YoY) Growth, and Customer Lifetime Value (CLV).
- Perform Recency, Frequency, Monetary (RFM) analysis to segment customers into actionable marketing groups.
- Establish a robust data pipeline that moves from raw transactional logs to an interactive PowerBI dashboard tailored for Sales Directors and Regional Managers.

2. Data Requirements & Architecture

2.1 Data Sources & Extraction

The project relies on synthesizing transactional order data with customer and product catalogs:

- **Standard Dataset Location:** High-quality sales datasets are widely available on Kaggle. Excellent choices include the "**Global Superstore Dataset**" (for retail/B2B) or the "**Brazilian E-Commerce Public Dataset by Olist**" (for deep e-commerce sales).
- **Data Types:**
 - * *Transactions:* Order_ID, Order_Date, Ship_Date, Customer_ID, Product_ID, Sales_Amount, Discount, Profit, Quantity.
 - *Product Catalog:* Product_ID, Category, Sub_Category, Product_Name, Base_Cost.
 - *Geography/Customer:* Customer_ID, Customer_Name, Segment (Consumer, Corporate), City, State, Region.

2.2 Data Preprocessing (Excel & Python)

Sales datasets often require cleaning to handle returns, missing geographic data, and date formatting.

- **Initial Screening (Excel):** Use Excel to spot-check the raw files. Apply conditional formatting to highlight negative `Sales_Amount` values (which typically indicate refunds/returns) to decide if they should be separated from gross revenue calculations.
- **Data Wrangling (Python / Pandas):**
 - **Datetime Indexing:** Convert `Order_Date` and `Ship_Date` into Pandas datetime objects to extract `Year`, `Quarter`, `Month`, and `Day_of_Week`.
 - **Handling Anomalies:** Identify and treat extreme outliers in the `Quantity` or `Discount` columns that could skew average order value calculations.
 - **Feature Creation:** Calculate the `Net Profit Margin` for each transaction directly in Pandas before loading it into the database.

2.3 Data Storage & Database Design (MySQL)

Cleaned transactional records will be stored in a relational database for structured, high-volume querying.

- **Schema Design:** Implement a classic Star Schema featuring a central `Fact_Sales` table connected to dimension tables: `Dim_Customer`, `Dim_Product`, `Dim_Geography`, and `Dim_Time`.
- **SQL Aggregations:**
 - Use `GROUP BY` and `SUM()` to calculate total revenue and profit by `Product_Category` and `Region`.
 - Utilize Window Functions (`SUM() OVER(PARTITION BY Region ORDER BY Order_Date)`) to calculate running Year-to-Date (YTD) totals for each specific sales territory.

3. System Features & Analytical Pipeline

3.1 Feature Engineering & Sales Metrics

Construct new variables using Python (`NumPy`, `Pandas`) to evaluate business performance:

- **Average Order Value (AOV):**
$$AOV = \frac{\text{Total Revenue}}{\text{Total Number of Orders}}$$
- **RFM Calculation:** Aggregate data at the `Customer_ID` level to calculate:
 - *Recency*: Days since the customer's last order.
 - *Frequency*: Total count of distinct orders.
 - *Monetary*: Total lifetime spend of the customer.

3.2 Exploratory Data Analysis (EDA)

Generate visual insights into sales behavior using Python (`Matplotlib`, `Seaborn`):

- **Pareto Charts (80/20 Rule):** Visualizing which 20% of products or customers are generating 80% of the total revenue or profit.
- **Time-Series Line Charts:** Plotting monthly sales revenue to visually demonstrate holiday seasonality or end-of-quarter spikes.
- **Correlation Heatmaps:** Mathematically mapping the relationship between **Discount** and **Profit** to prove at what discount threshold a product starts losing money.

3.3 Advanced Analytics (Machine Learning)

Utilize Python's **scikit-learn** to segment customers and forecast future sales:

- **Customer Segmentation (Unsupervised):**
 - *K-Means Clustering:* Feed the RFM scores into a K-Means algorithm to automatically group customers into distinct tiers (e.g., "Champions," "At-Risk," "Lost Cheap Customers"). Evaluate cluster quality using the Silhouette Score.
- **Sales Forecasting (Time-Series):**
 - *ARIMA or Prophet (Optional):* Train a time-series model on the aggregated weekly sales data to forecast the expected baseline revenue for the upcoming quarter, providing a target for the sales team.

4. Interactive Dashboarding (PowerBI)

The final deliverable is an operational PowerBI dashboard designed for the VP of Sales and territory managers.

- **Data Modeling:** Import the SQL schema and Python cluster outputs into PowerBI, establishing 1-to-Many relationships with the central Fact table.
- **DAX Calculations:** Create dynamic measures for **YoY Revenue Growth %**, **Profit Margin %**, and **Total Customers**.
- **Visual Interface:**
 - **Executive KPI Ribbon:** High-level cards displaying Total Sales, Total Profit, AOV, and Total Orders.
 - **Geospatial Sales Map:** A map visual plotting regions or cities, where bubble size represents Revenue and color saturation represents Profit Margin (highlighting regions that sell a lot but at a loss).
 - **Customer Segment Treemap:** Visualizing the proportion of the customer base that falls into each K-Means generated segment.
 - **Product Matrix:** A detailed table listing top products, conditionally formatted with data bars to show sales volume.
 - **Interactive Slicers:** Allow executives to instantly filter the dashboard by **Year/Quarter**, **Region**, and **Product Category**.

5. Project Delivery & Evaluation Structure

5.1 Directory Organization

The final project repository must be logically partitioned to facilitate technical review:

- **/data:** Raw transactional CSV extracts and the processed, segmented datasets.
- **/scripts:** Python modules (.ipynb) separated logically by workflow:
 - 1_data_cleaning_and_sql_formatting.ipynb
 - 2_eda_and_sales_metrics.ipynb
 - 3_rfm_calculation_and_kmeans_clustering.ipynb
- **/sql:** All DDL (table creation) and DML (aggregations, running totals, views for PowerBI) scripts.
- **/reports:** The finalized .pbix PowerBI dashboard file.

5.2 Code Quality & Maintainability

- **Documentation:** Python scripts must contain clear markdown cells explaining the business logic, especially regarding how returns (negative sales) were handled and the methodology for selecting the number of clusters (\$K\$) in the K-Means algorithm.
- **Data Integrity:** Strict enforcement of SQL Primary Keys (**Order_ID**) and Foreign Keys to ensure data consistency between the dimensional and fact tables, mimicking an enterprise data warehouse environment.